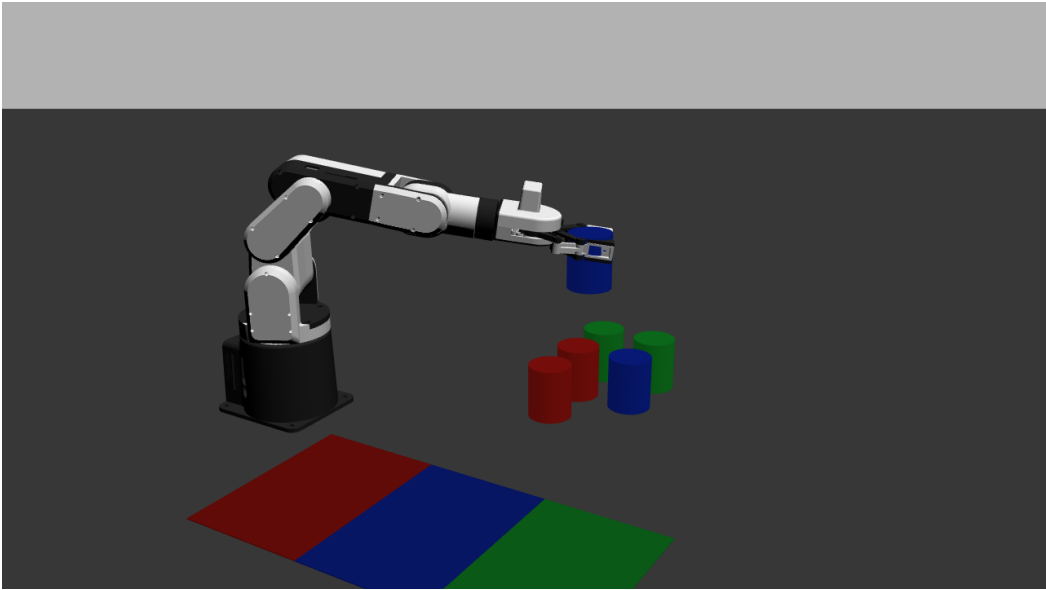


EXPERIMENT 3

Tabletop Object Automatic Sorting and Organization

Personal Laboratory Report · mechArm 270 Simulation Debugging Record

Personal laboratory report



Name	Yang Muqing
Student ID	24020036076
Group and Course	Group 17 · Robotics Integration Group Project I

ROS 2 HUMBLE · GAZEBO · mechARM 270

SEPTEMBER 2026

GitHub: <https://github.com/9161942-boop/robotics-experiment-3>

1 Basic Experimental Requirements

The objective of Experiment 3 is to make the robot arm automatically sort tabletop objects according to visual results. The program obtains object classes, confidence scores, and positions from camera images and detection arrays, maps the detection-box centers to a fixed pickup grid, selects the corresponding sorting pad by class, and then completes approach, grasp, lift, transport, release, and result logging in sequence. After one target is processed, the task node continues with the remaining targets.

Acceptance covers automatic decisions, exception branches, and evidence completeness. During execution, the system determines the grid and sorting bin automatically. Low-confidence detections, out-of-grid detections, duplicate occupancy, and unreachable targets produce explainable skip or stop results. Recognition, grasping, placement, and the final state are checked through logs, images, and pose records. The two-class, six-object simulation baseline requires at least five objects to be sorted correctly.

1.2 Experimental Environment and Parameters

This personal work was completed in Ubuntu 22.04, ROS 2 Humble, and Gazebo Classic 11. Python/rclpy was used to organize the scene-monitoring, task-planning, and execution nodes. The robot was a six-degree-of-freedom mechArm 270 model, with the gripper controlled through ros2_control. The top-view camera produced 640 x 480 images, and detections entered the task node as Detection2DArray messages.

Item	Final adopted setting
Scene	0.72 m x 0.52 m tabletop; 3-row by 2-column six-cell pickup area; red, blue, and green cylinders
Sorting pads	Red (0.14, -0.12), blue (0.26, -0.12), and green (0.38, -0.12) m, arranged horizontally
Recognition rule	Confidence threshold 0.80; nearest-grid distance threshold 48 px
Teaching record	manual_animation.json; 27 waypoints; 34-step automatic playback; 2.0 s per step
Acceptance records	Task CSV, execution CSV, exception logs, GUI process images, and final poses

1.3 Work Completed in This Experiment

I participated in adjusting the simulation scene, teaching motions manually, organizing the automatic sequence, checking grasp attachment, and reviewing the results. Object-level records for the six red and blue targets show 6/6 recognition, grasping, and placement, with a final status of passed. For the red-blue-green extension, I aligned the sorting pads, taught waypoints for the green targets, and organized the 34-step playback sequence. The archived materials currently contain scene and sequence evidence; the object-level success rate for the three-class extension will be calculated after an independent CSV archive is completed.

2 Personal Responsibilities

My work focused on turning detection and task-planning results into stable, verifiable robot-arm motions. I followed the sequence of environment startup, manual teaching, parameter tuning, automatic playback, and evidence review.

- Checked the startup relationships among the Gazebo world, controllers, camera, and task nodes, and kept the control interfaces and process instances consistent.

- Used the manual control panel to adjust joints, the gripper, and motion duration, and recorded the above, pick, lift, and release waypoints for every grid cell.
- Checked object geometry, sorting-pad locations, the confidence threshold, and pixel-grid parameters so that the image, planning, and execution layers used the same coordinate relationship.
- Organized the automatic playback subsequences for the six targets, checked the action semantics of the pick and release waypoints, and reviewed `manual_animation.json`.
- Combined attached, release, settled, and final-pose evidence to determine whether an action succeeded and to establish a verifiable acceptance basis.
- Used the group-provided detection results to check the transfer of grid IDs, classes, and task queues through the execution node, and matched those checks with the motion logs.

2.1 Scene and Startup Organization

I first checked that the tabletop dimensions, six cylinders, and three sorting pads in the scene file were all within the camera view. I then confirmed that the world coordinates matched the pixel grid used by the task node. During manual debugging I started only the world, controller, and manual panel. For automatic acceptance I started the task player and ensured that Gazebo master, the controller, and the player each had only one instance.

2.2 Manual Teaching and Waypoint Recording

Using the joint sliders and gripper parameters, I located the safe height, approach position, grasp position, lift position, and release position point by point. I used a single-variable procedure: each test changed only one target or one action stage, and I saved the waypoint after confirming the relative position of the end effector and cylinder. The waypoint records also identified the approach, closing, lifting, and release stages, which made later diagnosis easier.

2.3 Parameter and Grasp Verification

I focused on whether the cylinder was between the two fingers, whether the target rose with the end effector after closing, and whether X/Y remained consistent before and after release. For the logical attachment used in the simulation, I treated a target height above 0.075 m as the visual evidence condition and checked the attached event at the same time. These two checks distinguish a closed gripper from a successful grasp.

2.4 Automatic Sequence and Log Organization

I first ran one target, then the complete subsequence for one target, and only then combined all six targets. A waypoint name containing pick triggered grasping, a waypoint name containing release triggered release, and the other waypoints represented motion only. After automatic execution, I compared object IDs, grid IDs, classes, events, and final poses, and organized CSV and exception records that could be cross-checked against the screenshots.

2.5 Integrated Acceptance

Finally, I used the six-target red-blue scene for the object-level baseline check, and separately checked the sorting-pad arrangement and automatic playback order in the red-blue-green extension. The extension results were archived separately as scene evidence, interface behavior, and object-level data. The 6/6 red-blue baseline result is reported independently.



Figure 1 Position of my work in the chain from manual teaching to object-level acceptance

3 Project Implementation Principles

3.1 System Components and Data Flow

From the perspective of personal debugging, the system can be divided into five layers: scene, perception, planning, execution, and evidence. Gazebo provides the tabletop, cylinders, sorting pads, and robot-arm model. The monitoring node publishes Image and Detection2DArray messages. The task node performs confidence filtering, grid matching, class lookup, and queue deduplication. The execution node reads the queue and performs grasping and release. The GUI, event logs, CSV files, and final poses together form the acceptance evidence.

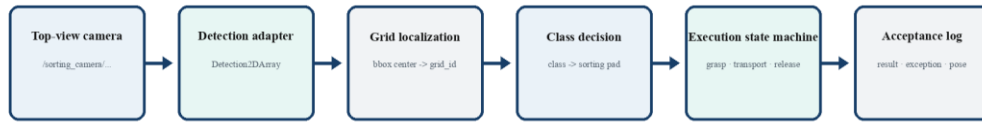


Figure 2 Connections among recognition, grid localization, classification, and execution modules in the simulation system

3.2 Grid Localization and Classification Decision

The core relationship I helped verify was detection-box center -> nearest grid -> class -> sorting pad. The task node projects the detection-box center into image coordinates and compares it with the fixed points in a 3-row by 2-column grid. A grid_id is generated only when the nearest-grid distance is no more than 48 px. The classes red, blue, and green are mapped through an explicit configuration table to red_sort_bin, blue_sort_bin, and green_sort_bin. Unknown classes, low-confidence results, and repeated detections in the same grid are marked skipped with the corresponding reason.

Six pickup cells in the red-blue baseline

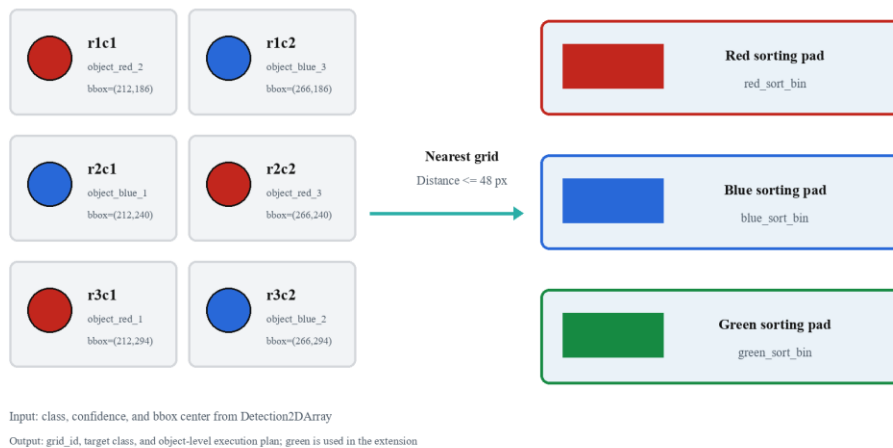
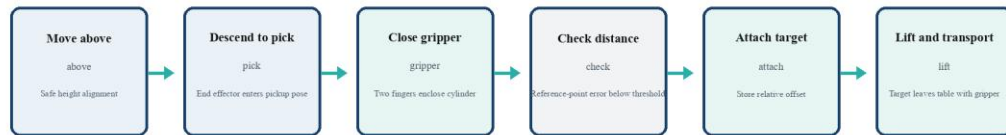


Figure 3 Relationship among the six grids, object classes, and sorting pads that I helped verify

3.3 Grasping and Transport Motion Chain

A reproducible grasp is divided into six stages: move to the safe above height, descend to pick, close the gripper, check the distance between the end effector and target, confirm attachment, and lift before transporting the target above the sorting pad. During transport, the target-to-gripper relative offset is stored and updated so that the target continues to follow the end effector.



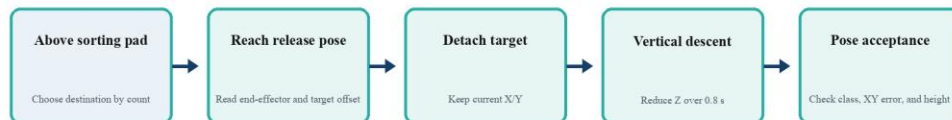
Success check: gripper closed, target between fingers, and target Z above the tabletop height.

Failure handling: reject attachment when the distance threshold is exceeded; record an exception instead of claiming success.

Figure 4 Approach, closing, checking, attachment, and lifting actions that I debugged

3.4 Vertical Release and Destination Check

During release, the current end-effector position is read, X/Y alignment is maintained, and the end effector descends along Z. After attachment is released, the system waits for the target to settle and reads the final pose. Multiple targets of the same color are assigned different destinations so that the placement order remains clear.



Why not teleport: the release start is determined by the current gripper pose, avoiding a lateral jump to the sorting pad center.

No same-color overlap: each color keeps separate sub-destinations for its first and second target.

Figure 5 Vertical detachment and destination-check procedure

3.5 Waypoint Semantics and State Evidence

manual_animation.json stores the verified waypoints and action order, while the task node makes the class decision. The player triggers the corresponding semantics when a waypoint name contains pick or release; the other waypoints represent motion. A task is considered passed only when the GUI geometry, state feedback, attached/release events, and final-pose check agree. These four types of evidence jointly confirm the result.



Figure 6 Joint, gripper, and waypoint lists in the manual control panel



Figure 7 Automatic playback subsequences for the six targets

4 Problems Encountered and Solutions

This section reviews the debugging process problem by problem. Each item records the actual screen or log observation, the corresponding inspection point, the specific waypoint, parameter, or node that was changed, and the verification condition after the change. The object-level conclusion for the red-blue baseline follows mecharm_sorting_execution.csv; the three-class extension includes only results supported by existing scene and sequence evidence.

4.1 Default Pose Caused Gripper Misalignment

Observed behavior: During the first playback of a green target in the three-class scene, the robot arm moved from above_green_1 down to pick_4, but the gripper center did not lie on the cylinder centerline. The right fingertip touched the cylinder edge first. above_green_2/pick_5 showed the same tendency. Although the gripper closed, the cylinder was not stably held between the two fingers.

Cause analysis: I paused at the two waypoints in manual_control_panel, dragged the joint sliders one by one, and compared the end-effector position with the six pickup points in sorting_scene.yaml and the top-view image. The original default pose had been calibrated for fixed points A-B. Reusing it for the green grid cells made the approach

direction inconsistent with the cylinder center. The problem was therefore not the gripper opening, but the lack of independent above/pick pose calibration for each grid cell.

Handling: I used the already verified red-blue motion sequence as the template and re-saved two waypoint pairs, `above_green_1 -> pick_4` and `above_green_2 -> pick_5`. The above stage kept the gripper opening at 0.15 and contact-pad force at -4.0; the pick stage switched the gripper to -0.31, and each action duration was fixed at 2.0 s. I changed one waypoint pair at a time, executed it step by step in the GUI, wrote it back to `manual_animation.json`, and finally rebuilt the 34-step sequence.

Verification: In the single-target test, the two fingers approached the cylinder from both sides at the same time, and no obvious side-slip occurred after closing. I then played the six-target sequence from `manual_animation.json`; the green pick_4/pick_5 actions no longer collided with the edge. The evidence consisted of the manual-panel screenshot, the six-target sequence figure, and the Gazebo recording. I did not extend this evidence into a three-class object-level success rate.

4.2 Gripper Closed but Object Stayed on the Table

Observed behavior: In one blue-cylinder test, the log already showed `pick` and `close_gripper`, but the cylinder remained on the table at approximately $z = 0.0275$ m. After the arm lifted, the space below the gripper was empty. Judging only from gripper closure would have incorrectly classified this action as successful.

Cause analysis: I split the action into closing, distance checking, and lifting and found that the original player published only gripper and arm `JointTrajectory` commands. It did not preserve the relative pose between the target and the end effector. Before the fix, `sorting_executor.py` did not associate the target pose from `/gazebo/model_states` with the gripper pose from `/gazebo/link_states`, so a command being issued did not mean that the target was attached.

Handling: In `sorting_executor.py` I added grasp-object selection and a distance check, with a grasp-position tolerance of 0.05 m. After confirming the object, the executor stored `attach_offset` and updated the selected model pose through `/gazebo/set_entity_state` at approximately 0.06 s intervals. The state machine also wrote a `grasp_attached` event. If no nearby target was found, it entered the failed/exception branch; a target height above 0.075 m was used as the visual evidence condition.

Verification: After rerunning the single-target test, `blue_cylinder_grasp_verified.png` clearly showed the cylinder between the two fingers and above the tabletop. The corresponding CSV contained `grasp_attached`, `lift`, `release`, and `object_complete` events in order, all with status passed. This verifies that the target actually rose with the end effector rather than merely confirming that the gripper closed.

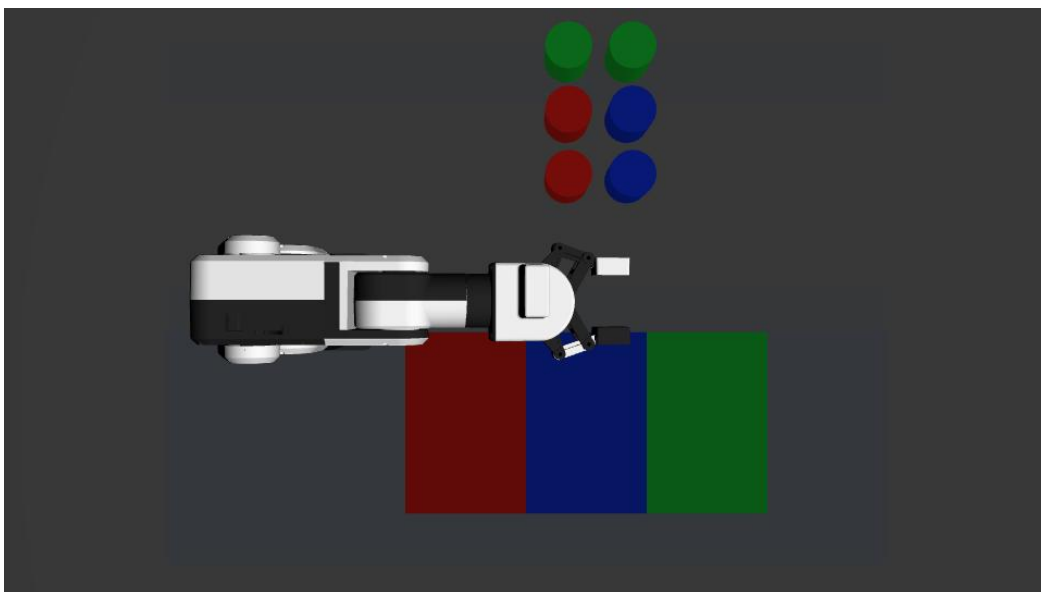


Figure 8 Verification image showing the closed gripper holding the target between the fingers and lifting it from the table

4.3 Release Jump and Overlap of Same-Color Targets

Observed behavior: In the first release test, the target was written directly to the center of the sorting pad, so the image did not show a continuous alignment, detachment, and falling process. Two targets of the same color also

landed at exactly the same position. The later target covered the earlier one, making it impossible to determine from the final image whether the targets had been completed one by one.

Cause analysis: After checking the release stage, I found that one `SetEntityState` call changed the target X, Y, and Z simultaneously. The target therefore jumped sideways between adjacent frames. The original playback sequence also had no separate destination index for same-color targets. Even when the final coordinates looked correct, the release process did not satisfy the acceptance requirements for continuous motion and reproducible results.

Handling: In the executor I created separate destination lists for red, blue, and green. Red used (0.115, -0.15), (0.14, -0.12), and (0.165, -0.09); blue used (0.235, -0.15), (0.26, -0.12), and (0.285, -0.09); green used (0.34, -0.02) and (0.38, 0.02). During release, X/Y alignment was maintained, the target was detached before descending along Z, and the system waited for stability after opening the gripper. The verification stage checked that the XY error was no more than 0.07 m; otherwise it recorded release pose outside destination.

Verification: `release_stages.png` continuously showed four stages: reaching the release pose, detaching, vertical descent, and settling. The final positions of same-color cylinders were separated. The CSV release record preserved the destination coordinates and was followed by `object_complete`. This change therefore solved both the visual jump and the overlap of same-color targets.

4.4 World Coordinates and Pixel Grid Out of Sync

Observed behavior: After the scene coordinates were adjusted, the task node recognized only a few targets and the complete plan contained gaps. The top-view image showed all six cylinders on the table, but the target count in the task log did not match the image. The missing objects caused later grasp sequences to be incomplete; the robot-arm motion itself had not failed.

Cause analysis: I compared the positions of the three rows of targets in the world, the projection formula in `sorting_scene_monitor.py`, and the pixel grid used by `sorting_task.py`. The scene had changed while the old grid parameters were still being loaded. The current projection maps world-y to the image horizontal coordinate and world-x to the image vertical coordinate. If the old row coordinates are used, the detection-box centers are assigned to the wrong grid or classified as outside the grid.

Handling: I recalculated the three row pixel coordinates as 186, 240, and 294 according to the new projection relationship, while retaining the column coordinates 212, 266, and 320. I fixed `grid_tolerance_px` at 48, cleared old instances, and restarted monitor, task, and executor. To prevent a moved target from triggering a new plan, I retained detection-signature deduplication and the `plan_locked` guard under `execute_motion`.

Verification: After restart, `grid_mapping.png` showed a one-to-one correspondence between detection-box centers and the six grids, and `grid_id` values in the log were no longer duplicated or missing. The red-blue baseline CSV recorded `count=6` and `success_count=6` in `task_result`, with a unique grid assigned to each object. This shows that the coordinate correction reached both the task queue and the final result.

4.5 Repeated Launches Prevented Motion Commands

Observed behavior: The interface looked normal, but the robot arm did not respond after the player sent a motion command, or the same motion was triggered twice. The terminal showed duplicate output for the same stage. If I continued adjusting waypoints at that point, I could mistake a process conflict for a pose-parameter error.

Cause analysis: I checked the world, controller, and player processes and found that an old Gazebo master and a new instance were running simultaneously. The manual control panel could also be connected to the old player. Multiple instances listened to the same control topics, so one command could be consumed twice or sent to a controller that was already invalid.

Handling: I first terminated the old `gzserver`, `gzclient`, and player processes, leaving only one world instance. I then started Gazebo, the controller, and `manual_control_panel` in the order specified by `start_contact_grasp.sh`. Automatic mode started only one `sorting_gazebo.launch.py`. The task node's detection-signature deduplication and `plan_locked` flag prevented the same plan from being submitted to the executor again.

Verification: After the environment was cleaned, the controller received only one set of motion commands. The single-target test progressed from above through pick and lift to release, and the 34-step sequence was no longer triggered repeatedly. During verification I checked the GUI, `sorting_task/execution_state`, and CSV timestamps together, confirming that the problem came from duplicate instances rather than the waypoints themselves.

4.6 Node Startup Log Error

Observed behavior: The task node exited immediately after startup. The terminal reported incompatible log-call arguments, so the node never entered detecting or plan_ready. The Gazebo scene was still visible, but there was no task state or object-level result; subsequent grasping could not be treated as a valid experimental record.

Cause analysis: I traced the error to the way the Humble Python logger was called. The old code passed extra arguments to the logging function, and the exception occurred before the state machine actually entered a task state. By checking the node exit position and the state topics after startup, I confirmed that this was not a detection or robot-control problem but a logging-call error during node initialization.

Handling: I changed the affected log messages to Humble-compatible f-string formatting, rebuilt the workspace, and restarted the unified Launch from a clean terminal. After startup I first checked that sorting_task/state, sorting_task/result, and sorting_task/execution_state existed. I then ran the single-target and complete sequences so that a process that had not started successfully would not be included in acceptance.

Verification: After the fix, the node remained active and continuously published detecting, plan_ready, execution-stage, and task_passed states. The CSV also recorded plan, grasp_attached, release, and task_result. Grasping issues could therefore be analyzed separately in a stable startup environment, and the log error no longer hid the task's execution result.

Problem	My evidence basis	Post-fix check
Pose misalignment	Relative position of end effector and target	Grasp distance and alignment image
Not attached	Target height and attached event	Target rises with the gripper
Release anomaly	X/Y and Z changes before and after release	Vertical descent and separated destinations
Grid error	Projected pixels and grid_id	Unique ID for each target
Runtime conflict	Number of world/controller instances	Single-instance playback
Node exit	Terminal error location	Node continuously publishes state

5 Simulation Results and Personal Summary

5.1 Red-Blue Baseline Object-Level Results

The object-level CSV for the six red and blue targets is the main quantitative basis for acceptance in this experiment. All six targets had a recognition confidence of 0.99, and both grasping and placement were completed, with a final status of passed and success_count=6. This satisfies the task requirement of correctly sorting at least five objects in the red-blue baseline.

Class	Count	Conf.	Grasp	Place	Result
Blue cylinder	3	0.99	3/3	3/3	Pass

Class	Count	Conf.	Grasp	Place	Result
Red cylinder	3	0.99	3/3	3/3	Pass
Total	6	6/6	6/6	6/6	Pass

5.2 Boundary of the Three-Class Extension Results

I participated in completing the red-blue-green scene, aligning the sorting pads, and organizing the automatic playback sequence. The extension demonstrates that the added green class can reuse the detection message, grid localization, class mapping, and execution interfaces. The current archive contains scene, waypoint, and playback evidence; the corresponding success rate will be calculated after the three-class object-level CSV is archived.



Figure 9 Connections among recognition, localization, classification, and execution in the three-class extension

5.3 Evidence Chain and Debugging Tools

The GUI was used to check the geometric relationship among the end effector, gripper, and target. State feedback identified whether the system was scanning, planning, executing, or releasing. Event logs confirmed whether attached, release, and settled occurred in order. The final pose was used to judge the destination, and the object-level CSV aggregated these observations by object_id. Only after the multiple evidence layers agreed could I distinguish an action being completed from the task being successful.



Acceptance rule: one animation or one 'complete' log alone is insufficient; process, events, and final destination must corroborate one another.

Figure 10 Evidence chain of the process view, state, events, and CSV that I organized

5.4 Personal Summary

This experiment showed me that robot-arm simulation requires joint motion, scene models, control interfaces, task logic, and acceptance evidence to remain consistent at the same time. An action has a reproducible basis for success only when the target is selected correctly, the gripper-target relative pose satisfies the condition, the target height changes, the event order is complete, and the final pose lies inside the sorting region. This changed the way I

judge simulation results: I now treat the image, state, and data as different observations of the same event rather than looking only at whether the arm completed a trajectory.

During debugging, I gradually developed a fixed troubleshooting order. First, I narrow the problem to one stage: scanning, grid localization, approach, grasping, transport, release, or acceptance. Second, I reproduce the behavior with one target and the manual panel, recording the end-effector position, target position, gripper state, and current waypoint. Third, I change only one variable at a time, such as the pixel grid, grasp height, gripper opening, release height, or action duration. Fourth, I use event logs, final coordinates, and the complete sequence to review the change. Grid misalignment, release jumps, unattached targets, and duplicate launches were all located using this procedure.

- Use staged reasoning to analyze motion: first confirm which state the task entered, then check that state's inputs, outputs, and exit conditions.
- Use geometric relationships to verify grasping: observe the finger-pad center, cylinder center, target height, and relative pose together to confirm that the motion trajectory agrees with the object state.
- Use minimal experiments to reduce interference: test one point first, then one target subsequence, and finally the complete sequence, reducing the difficulty of distinguishing multiple simultaneous problems.
- Use object-level records to close the loop: associate object_id, grid_id, class, events, final pose, and error code so that every target can be reviewed independently.
- Use a clean environment for reproduction: control the number of Gazebo world, controller, and player instances so that every test starts from the same conditions.

This experiment also made me more familiar with interface responsibilities among modules. The task node generates object order and sorting targets, the execution node performs actions and returns feedback, and the scene and logs provide the basis for review. My work lies between task-planning results and simulation acceptance: I must turn abstract task items into executable poses, waypoints, and state checks. This made me pay more attention to the completeness of names, parameter records, log fields, and image captions, because they directly affect collaboration among group members and later reproduction.

The logical attachment in the simulation improved motion repeatability and also made the difference between simulation and a real robot clear to me. A real system would require new calibration of the camera mapping, gripper friction, approach height, closing speed, release wait, and emergency-stop distance. When moving to the real robot, I will keep the sequence of low-speed single-target test, recognition check, grasp check, release check, adding a second class, and expanding to six targets. I will save object-level records at every step before deciding whether to enter the next stage.

6 Conclusion

I completed the main personal work on the execution side of the Experiment 3 simulation and linked my contribution to four layers: scene, waypoints, state, and evidence. Through scene adjustment, manual teaching, motion repair, automatic playback, and object-level review, I participated in building a complete chain from task-planning results to robot-arm motion and then to the experimental conclusion. The object-level result for the six red and blue targets reached 6/6, and the three-class extension completed the integration of the scene, sorting pads, and automatic sequence. On the execution side, I used the group-provided detection results to check grid mapping, grasping, and release behavior.

6.1 Personal Work Completion

Work module	Completed content	Corresponding result
Scene and startup	Adjusted the six-cell tabletop, red-blue-green cylinders, and three sorting pads; checked startup relationships among the world, controller, camera, and player	Three-class scene loads stably and runs with one instance
Waypoints and motion	Established semantic waypoints such as above, pick, lift, and release, and organized the six-target subsequences with a unified action timing	manual_animation.json stores 27 waypoints and automatic playback has 34 steps

Work module	Completed content	Corresponding result
Grasp and release	Calibrated the end-effector reference point and added distance checking, target following, vertical release, and separate destinations for same-color targets	Targets leave the table, follow the arm during transport, and land in the corresponding sorting pads
Problem repair	Handled pose offset, grid desynchronization, repeated startup, node log errors, and release jumps	Single-target and full-sequence reruns passed
Result review	Cross-checked GUI, state, events, final poses, and object-level CSV	Red-blue baseline object-level result is 6/6

6.2 Skills Gained

I learned how to coordinate Gazebo scene parameters with ROS 2 controllers and how to inspect the same action from four perspectives: world coordinates, pixel grids, end-effector pose, and object state. Through the manual control panel and JSON waypoint file, I became familiar with organizing motions from a single-point test to a complete sequence. Through state topics, event logs, and object-level CSV files, I can now split one run into stages that are easy to locate and compare.

In collaboration, I gained a deeper understanding of the importance of unified interfaces and naming. The `object_id`, `grid_id`, and `target_bin` produced by the task node must correspond to the waypoints, logs, and final poses in the execution node. The images and tables in the report must also lead back to a specific object. This record-keeping method improves problem localization and allows personal debugging results to directly support group acceptance.

6.3 Future Improvement Plan

For future real-robot work, I will first complete camera homography calibration and mark the tabletop grid, then use a single target to verify recognition, PICK, lift, PLACE, and HOME. After confirming the grasp height, gripper opening, release wait, and emergency-stop strategy, I will add a second class and expand to six targets. At every stage I will continue recording the true class, predicted class, confidence, grid ID, grasp result, placement result, and error code so that the records can be compared with the current simulation fields.

From a personal-method perspective, I will keep the habits of stage localization, single-variable changes, minimal-case reproduction, and complete-data review. I will also add run IDs, log rotation, continuous-frame stability checks, and exception-recovery records so that future experiments have better reproducibility and safety.

6.4 GitHub Synchronization Record

The Experiment 3 project, evidence, and reports that I helped organize have been synchronized to the GitHub repository: <https://github.com/9161942-boop/robotics-experiment-3>.

The repository keeps the ROS 2 project, Gazebo scene, `manual_animation.json`, object-level execution records, process screenshots, and the group and personal reports. The two Gazebo simulation demonstration videos are also stored in the `videos/` directory. The commit-history screenshot documents that the shared materials were synchronized to the public repository, and the problems, adjustments, and verification records in this personal report can be matched across the report, screenshots, logs, and repository directories.

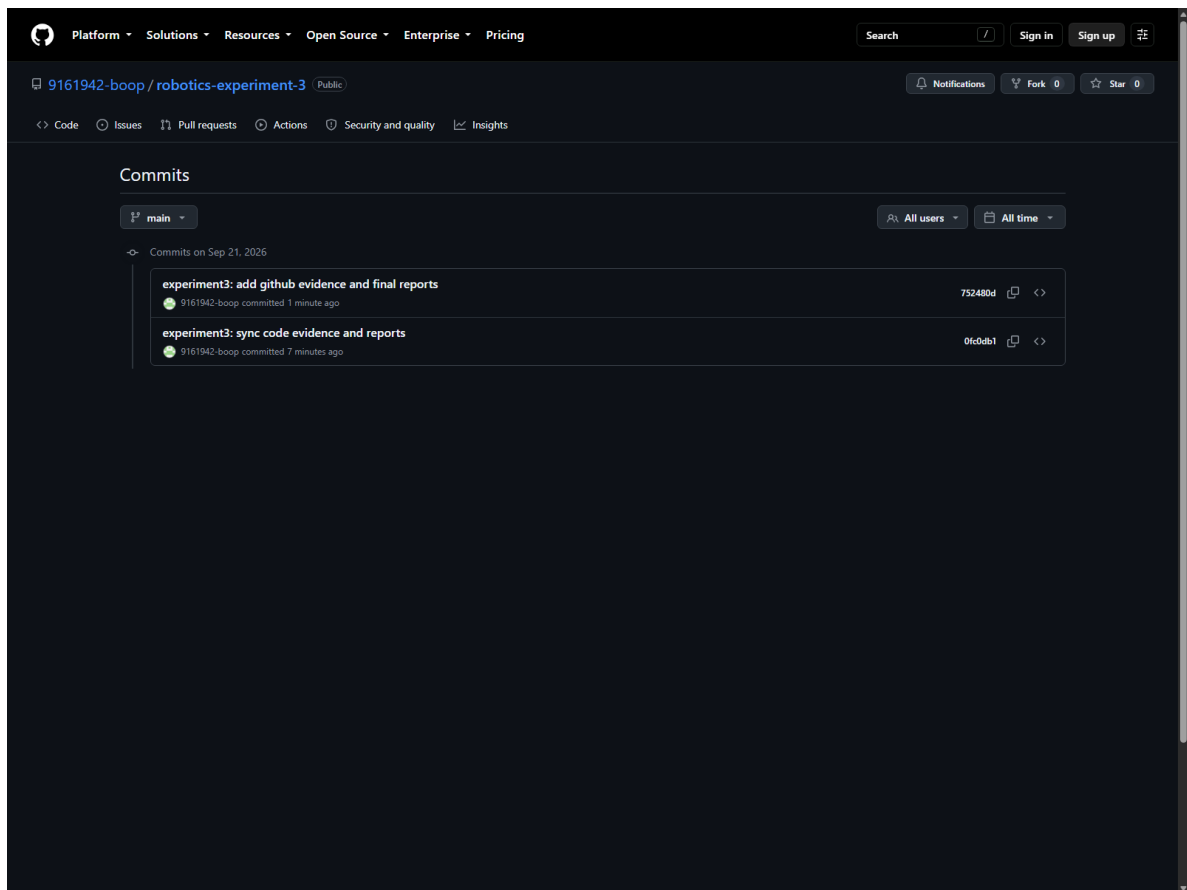


Figure 11 Commit history of the GitHub Experiment 3 repository corresponding to this personal report